

MIT 6.419x: 시간의 패턴을 읽다

Module 3 (Part A): Time Series Analysis

Contents

1	Module 3 (Part A). 시계열 분석 (Time Series Analysis)	2
1.1	1. 추세 (Trend)와 계절성 (Seasonality) 제거	2
1.2	2. 자기회귀 이동평균 모델 (ARMA / ARIMA)	3
1.3	3. 칼만 필터 (Kalman Filter)	4
2	실전 시나리오: 넥슨 게임 지표 분석	4
3	자주 묻는 질문 (FAQ)	5

Course Structure & Current Focus

- Module 1 & 2: Static Data (정적 데이터 분석)
- **Module 3: Time Series & Spatial Data (현재 단원: 시공간 데이터)**
 - 3.1 Trend & Seasonality (정상성 만들기)
 - 3.2 ARMA / ARIMA (통계적 예측 모델)
 - 3.3 Kalman Filter (상태 추정 및 노이즈 제거)
 - 3.4 Spatial Statistics (Kriging)

1 Module 3 (Part A). 시계열 분석 (Time Series Analysis)

지금까지 우리는 데이터의 순서를 신경 쓰지 않았습니다(*i.i.d* 가정). 하지만 주가, 게임 접속자수, 서버 로그는 순서가 핵심입니다. 어제의 데이터가 오늘의 데이터에 영향을 미치는 자기상관(*Autocorrelation*)의 세계로 들어갑니다.

□ 개요 (Overview)

이 단원에서는 시계열 데이터를 분석 가능한 형태로 가공하는 분해(Decomposition) 기법, 과거 데이터를 기반으로 미래를 예측하는 ARIMA 모델, 그리고 노이즈가 섞인 실시간 데이터에서 진짜 상태를 추정하는 칼만 필터(Kalman Filter)를 다룹니다.

□ 핵심 용어 사전

용어 (Term)	직관적 의미 (Meaning)
Stationarity (정상성)	평균과 분산이 시간에 따라 변하지 않고 일정한 상태. (예측의 전제 조건)
Seasonality (계절성)	주말마다 접속자가 늘어나는 것처럼 주기적인 패턴.
Lag (시차)	현재 시점(t)과 과거 시점($t - k$) 사이의 간격.
White Noise	패턴이 전혀 없는 완전 무작위 잡음. (예측 불가능한 영역)
State-Space Model	관측된 데이터 뒤에 숨겨진 '진짜 상태'가 있다고 가정하는 모델.

1.1 1. 추세(Trend)와 계절성(Seasonality) 제거

□ 개념 1: 양과 꺾질 벗기기

한 줄 요약: 데이터에서 '뻗은 패턴(상승세, 주말 효과)'을 걷어내야, 진짜 중요한 '변화(신호)'가 보입니다. 이를 위해 데이터를 분해합니다.

1) 시계열 분해 (Decomposition)

$$Y_t = \underbrace{T_t}_{\text{추세}} + \underbrace{S_t}_{\text{계절성}} + \underbrace{R_t}_{\text{잔차}}$$

우리는 Y_t 에서 T_t 와 S_t 를 제거하여, 정상성(Stationarity)을 가진 잔차 R_t 만 남기고 싶어 합니다. 왜냐하면 통계적 모델은 "평균이 일정한 데이터"에서 가장 잘 작동하기 때문입니다.

2) 제거 방법

- **차분 (Differencing, I):** $Y_t - Y_{t-1}$. 오늘의 값에서 어제의 값을 뺍니다. 추세(기울기)를 제거하여 수평으로 만듭니다.
- **로그 변환:** 분산이 점점 커지는 경우(갈때기 모양), 로그를 씌워 변동 폭을 일정하게 맞춥니다.
- **계절 차분:** $Y_t - Y_{t-7}$ (일주일 주기인 경우). 이번 주 토요일 데이터에서 지난주 토요일 데이터를 뺍니다.

1.2 2. 자기회귀 이동평균 모델 (ARMA / ARIMA)

□ 개념 2: 관성(Inertia)과 충격(Shock)

한 줄 요약: "관성대로 가려는 성질(AR)"과 "외부 충격에 반응하는 성질(MA)"을 합쳐서 미래를 그립니다.

1) 구성 요소: ARIMA(p, d, q)

- **AR (p , AutoRegressive):** 자기회귀.

$$Y_t = \phi Y_{t-1} + \epsilon_t$$

"어제 접속자가 많았으니, 관성에 의해 오늘도 많을 거야."

- **I (d , Integrated):** 차분. 추세를 제거하기 위해 데이터를 몇 번 미분(뺄셈)했는가? (정상성 확보용)
- **MA (q , Moving Average):** 이동평균.

$$Y_t = \epsilon_t + \theta \epsilon_{t-1}$$

"어제 예측보다 사람이 갑자기 더 몰렸어(Shock ϵ_{t-1}). 그 여파가 오늘까지 이어질 거야."

2) 모델 선택

보통 ACF(자기상관함수)와 PACF(편자기상관함수) 그래프를 그려서 p 와 q 값을 결정합니다.

1.3 3. 칼만 필터 (Kalman Filter)

□ 개념 3: 네비게이션의 원리

한 줄 요약: "내 예측(계산)"과 "센서값(측정)"은 둘 다 부정확합니다. 이 둘을 적절히 섞어서(가중 평균) 최적의 "진짜 위치"를 찾아냅니다.

1) 상태 공간 모델 (State-Space Model)

- **Hidden State (x_t):** 진짜 값 (예: 유저의 실제 게임 실력). 눈에 안 보임.
- **Measurement (z_t):** 관측 값 (예: 승패 결과). 노이즈가 섞여 있음.

2) 작동 원리: 무한 루프

1. **Predict (예측):** 어제의 상태를 바탕으로 오늘의 상태를 추측합니다. (불확실성 증가)
2. **Update (보정):** 실제 데이터(z_t)가 들어옵니다. 내 예측과 실제 데이터 사이에서, 더 신뢰할 수 있는 쪽으로 값을 수정합니다. (불확실성 감소)

3) 장점

과거 데이터를 다 들고 있을 필요 없이, 직전 상태($t-1$)만 있으면 됩니다. 그래서 실시간 (Real-time) 시스템에 매우 유리합니다.

2 실전 시나리오: 넥슨 게임 지표 분석

□ Scenario 1: 주말 이벤트 효과 분석 (Decomposition)

토요일에 경험치 2배 이벤트를 했습니다. 접속자가 평일 대비 30% 늘었습니다. 이게 이벤트 덕분일까요, 아니면 원래 토요일이라서 그런 걸까요?

- **해결:** 시계열 분해를 통해 '계절성(토요일 효과)' 성분을 제거합니다.
- **결과:** 계절성을 뺐더니 상승분이 5%밖에 안 남았습니다. 이벤트 효과는 생각보다 미미했습니다.

□ Scenario 2: 매치메이킹 시스템 (Kalman Filter)

유저의 실력(MMR)을 측정해야 합니다. 고수도 운 나쁘면 지고, 초보도 버스 타면 이깁니다. 승패(Measurement)는 노이즈 투성이입니다.

- **해결:** 칼만 필터를 적용합니다.

- **과정:** 유저가 이겼을 때, MMR을 확 올리지 않고 "이 유저의 실력 불확실성(분산)"을 고려해 조금만 올립니다. 판수가 쌓일수록 불확실성이 줄어들며 '진짜 실력'에 수렴합니다. (TrueSkill 알고리즘의 기초)

3 자주 묻는 질문 (FAQ)

- Q1. 딥러닝(RNN/LSTM)이 ARIMA보다 무조건 좋은가요?** A. 아닙니다. 데이터가 적거나(수백 개), 패턴이 단순하고 주기적이라면 **ARIMA**가 훨씬 정확하고 설명력(Why)도 좋습니다. 반면, 비선형적이고 복잡한 패턴(언어, 불규칙한 로그)은 딥러닝이 강합니다.
- Q2. 칼만 필터는 예측 모델인가요?** A. 엄밀히 말하면 '추정(Estimation)' 모델입니다. 현재의 '진짜 상태'를 가장 잘 맞추는 것이 목표입니다. 하지만 추정된 상태를 바탕으로 다음 단계를 Predict 하므로 예측에도 쓰입니다.

Next Step: 시간(t)에 대한 분석은 마쳤습니다. 그런데 데이터에는 시간 뿐만 아니라 위치(x, y) 정보도 중요합니다. "강남구의 집값이 오르면 서초구도 오를까?" 다음 시간에는 공간적 상관관계를 다루는 **공간 통계(Spatial Statistics)**와 **크리깅(Kriging)**을 배웁니다.

Module 3 (Part A) 핵심 요약

- **Decomposition:** $Y = T + S + R$. 추세와 계절성을 제거해야 예측이 가능하다.
- **ARIMA:** 과거의 나(AR)와 과거의 오차(MA)를 합쳐 미래를 예측하는 통계적 도구.
- **Kalman Filter:** 예측(Predict)과 보정(Update)을 반복하며 노이즈 속에서 진짜 상태를 찾아내는 알고리즘. (실시간 처리에 강함)